Hospital Management System

Database class project, academic year 2023 /2024

An abstract

The Hospital Management System project aims to streamline hospital operations by digitizing patient records, appointment scheduling, medication management, and invoice processing.

Utilizing a relational database, this system focuses on ensuring data integrity and accessibility for healthcare professionals.

Key entities include patients, doctors, nurses, appointments, medications, and invoices, each with unique attributes and interrelations.

By implementing functional dependencies and normalization, the system ensures efficient data handling and supports diverse healthcare processes.

This project encapsulates innovative technology solutions to enhance healthcare administration and patient care.

Table of Contents

- **Abstract {2}**
- **Introduction {4}**
- **Project Requirements {5-9}**
  - **Entities and Attributes {5}**
  - **Relationships {7}**
- **Functional Dependencies {10}**
- **Database Design**
  - **Unified Modeling Language (UML) Diagrams {11}**
  - **Mapping for Project {12}**
- **BCNF {13}**
- **Tools used in the project {13}**
- **A description of how the project's GUI works {14}**

Introduction

The Hospital Management System project is designed to modernize and improve the efficiency of hospital operations.

As the healthcare sector evolves, the need for reliable and scalable digital solutions becomes increasingly crucial.

This system addresses critical aspects of hospital management, including patient care, appointment scheduling, medication distribution, and financial transactions.

It leverages a robust database architecture to manage complex relationships among different entities such as patients, doctors, nurses, and administrative staff.

Through a user-friendly interface, the system provides seamless access to patient histories, appointment details, and billing information, thereby enhancing the quality of healthcare services.

This report details the development process, from the initial requirements gathering to the implementation and testing of the Hospital Management System, highlighting its potential impact on healthcare management efficiency.

Project Requirements for the Hospital Management System:

Entities and Attributes:

Admin:

The Admin can access for everything, like Edit , Update , Delete and Add new admin.

Patient Entity:

PatientID (Primary Key): A unique identifier for each patient.

First Name: The first name of the patient.

Last Name: The last name of the patient.

DateOfBirth: Date of birth of the patient.

Gender: The gender of the patient.

Address: The address of the patient.

Phone: Contact phone number for the patient.

Email: Contact email address for the patient.

HistoryID: A unique identifier for the medical history of the patient (if applicable).

BloodGroup: The blood group of the patient.

Doctors Entity:

ID (Primary Key): A unique identifier for each doctor.

First Name: The first name of the doctor.

Last Name: The last name of the doctor.

Specialist: The medical specialty or specialization of the doctor.

Mobile: Contact mobile number for the doctor.

Email: Contact email address for the doctor.

DoctorPassword: Password for doctor authentication.

Address: The address of the doctor.

Nurses Entity:

ID (Primary Key): A unique identifier for each nurse.

Name: The name of the nurse.

NurseDutyHour (or Shift): Information about the nurse's duty hours or shifts.

Email: Contact email address for the nurse.

Mobile: Contact mobile number for the nurse.

Password: Password for nurse authentication.

Appointments Entity:

ID (Primary Key): A unique identifier for each appointment.

Date: The date of the appointment.

Time: The time of the appointment.

DoctorID (Foreign Key): References the doctor associated with the appointment.

PatientID (Foreign Key): References the patient who has the appointment.

Description: Additional description or notes about the appointment.

Medications Entity:

ID (Primary Key): A unique identifier for each medication.

Name: The name of the medication.

Dosage: Information about the dosage of the medication.

ExpiryDate: The expiry date of the medication.

Invoice Entity:

ID (Primary Key): A unique identifier for each invoice.

PatientID (Foreign Key): References the patient associated with the invoice.

Amount: The total amount mentioned in the invoice.

PaymentStatus: Indicates the payment status of the invoice.

Relationships:

Nurses and Patient:

Relationship (many-to-many): A Nurse can have multiple patients, and a patient can have more than one nurse.

Foreign Keys:

NurseID in the Patients entity (as a foreign key linking to the Nurses entity) PatientID in the Nurses entity (as a foreign key linking to the Patients entity).

Doctor and Appointment:

(One-to-Many) Relationship: A doctor can have multiple appointments, but each appointment is associated with one specific doctor. The DoctorID in the Appointments entity serves as a foreign key to establish this relationship.

Patient and Appointment:

(One-to-Many) Relationship: A patient can have multiple appointments, but each appointment is associated with one specific patient. The PatientID in the Appointments entity serves as a foreign key to establish this relationship.

Medications and Patient:

(Many-to-Many) Relationship: Patients can be prescribed multiple medications, and each medication can be prescribed to multiple patients. This relationship is often achieved through an association table (Prescription) with MedicationID and PatientID as foreign keys.

Invoice and Patient:

(Many-to-One) Relationship: Many invoices can be issued to one patient, but each invoice is associated with one patient. The PatientID in the Invoice entity is a foreign key that links each invoice to a specific patient.

Nurses and Patient Relationship (Many-to-Many):

Entities Involved: Nurses & Patients

Relationship Type: Many-to-Many → PatientNurseAssignments (Association Table)

Foreign Keys:

NurseID in the PatientNurseAssignments entity (as a foreign key linking to the Nurses entity)

Patient in the PatientNurseAssignments entity (as a foreign key linking to the Patients entity)

Doctor and Appointment Relationship (One-to-Many):

Entities Involved: Doctor & Appointments

Relationship Type: One-to-Many

Foreign Key:

DoctorID in the Appointments entity (as a foreign key linking to the Doctors entity)

Patient and Appointment Relationship (One-to-Many):

Entities Involved: Patient & Appointments

Relationship Type: One-to-Many

Foreign Key:

PatientID in the Appointments entity (as a foreign key linking to the Patients entity)

Medications and Patient Relationship (Many-to-Many):

Entities Involved: Medications & Patients → Prescription (Association Table)

Relationship Type: Many-to-Many

Foreign Keys:

MedicationID in the Prescription entity (as a foreign key linking to the Medications entity)

PatientID in the Prescription entity (as a foreign key linking to the Patients entity)

Invoice and Patient Relationship (Many-to-One):

Entities Involved: Invoice & Patient

Relationship Type: Many-to-One

Foreign Key:

PatientID in the Invoice entity (as a foreign key linking to the Patients entity)

Based on the details from the provided document, here is the textual representation of the relational model with indications of total (T) and partial (P) participation:

Doctors (T) --1-- has Appointments --M-- Appointments (T)

Every Doctor must have at least one Appointment.

Each Appointment must be associated with one and only one Doctor.

Patients (T) --1-- has Appointments --M-- Appointments (T)

Every Patient must have at least one Appointment.

Each Appointment must be associated with one and only one Patient.

Patients (P) --M-- Prescribed --M-- Medications (P)

A Patient can be prescribed multiple Medications.

A Medication can be prescribed to multiple Patients.

This relationship is managed through a Prescription (Association Table).

Nurses (P) --M-- Assigned to --M-- Patients (P)

A Nurse can be assigned to multiple Patients.

A Patient can be assigned to multiple Nurses.

This relationship is managed through a NursePatient (Junction Table).

Patients (T) --1-- Issued --M-- Invoices (T)

Each Invoice must be associated with one and only one Patient.

A Patient can have multiple Invoices.

Key Types:

Primary Key (PK): Used to uniquely identify each record within an entity.

Foreign Key (FK): A field that links to the primary key of another entity to establish relationships.

Functional Dependency

Patient Entity:

PatientID → [FirstName, LastName, DateOfBirth, Gender, Address, Phone, Email, HistoryID, BloodGroup]

HistoryID → [Medical History Details (if applicable)]

Doctors Entity:

DoctorID → [FirstName, LastName, Specialist, Mobile, Email, DoctorPassword, Address]

Nurses Entity:

NurseID → [Name, NurseDutyHour, Email, Mobile, Password]

Appointments Entity:

AppointmentID → [Date, Time, DoctorID, PatientID, Description]

DoctorID → [Doctor Details]

PatientID → [Patient Details]

Medications Entity:

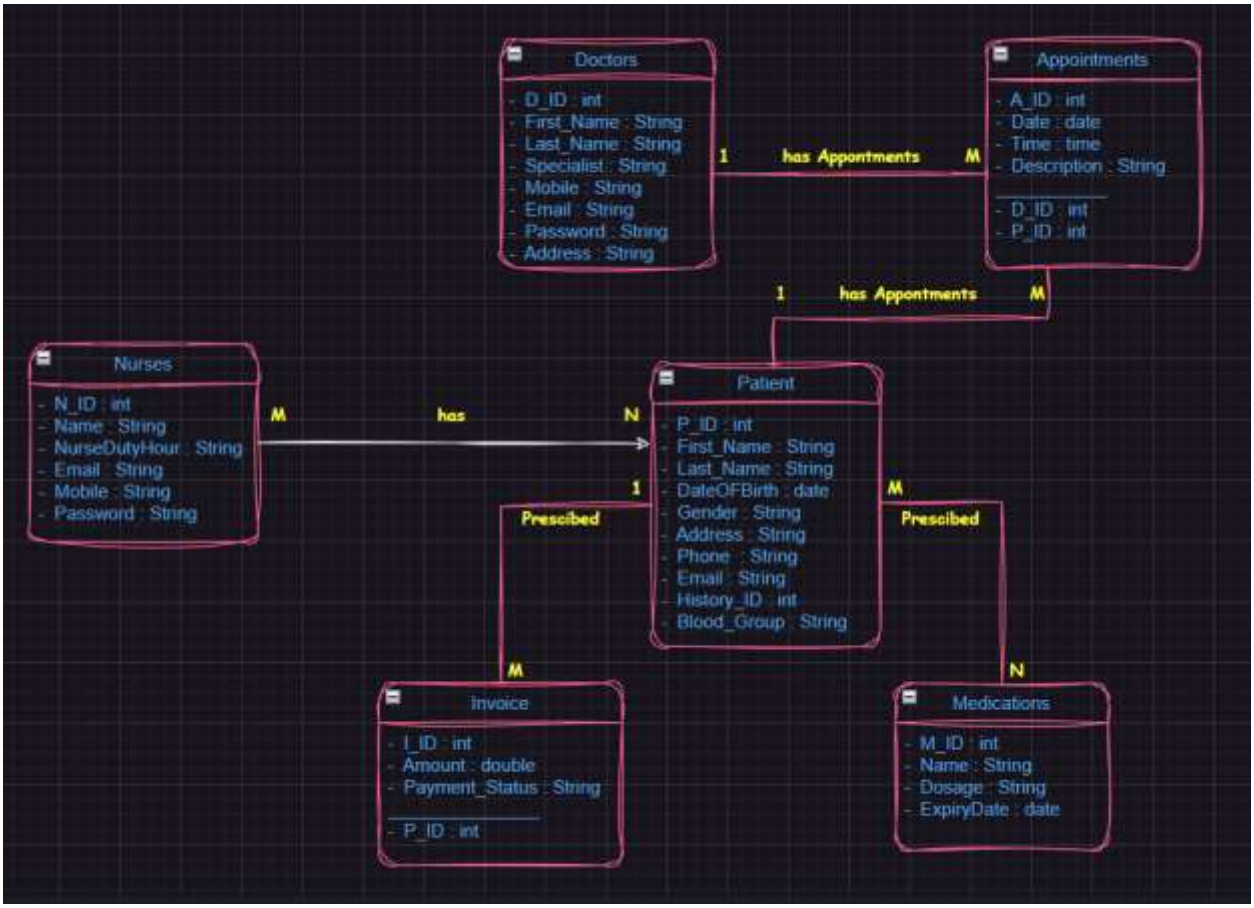
MedicationID → [Name, Dosage, ExpiryDate]

Invoice Entity:

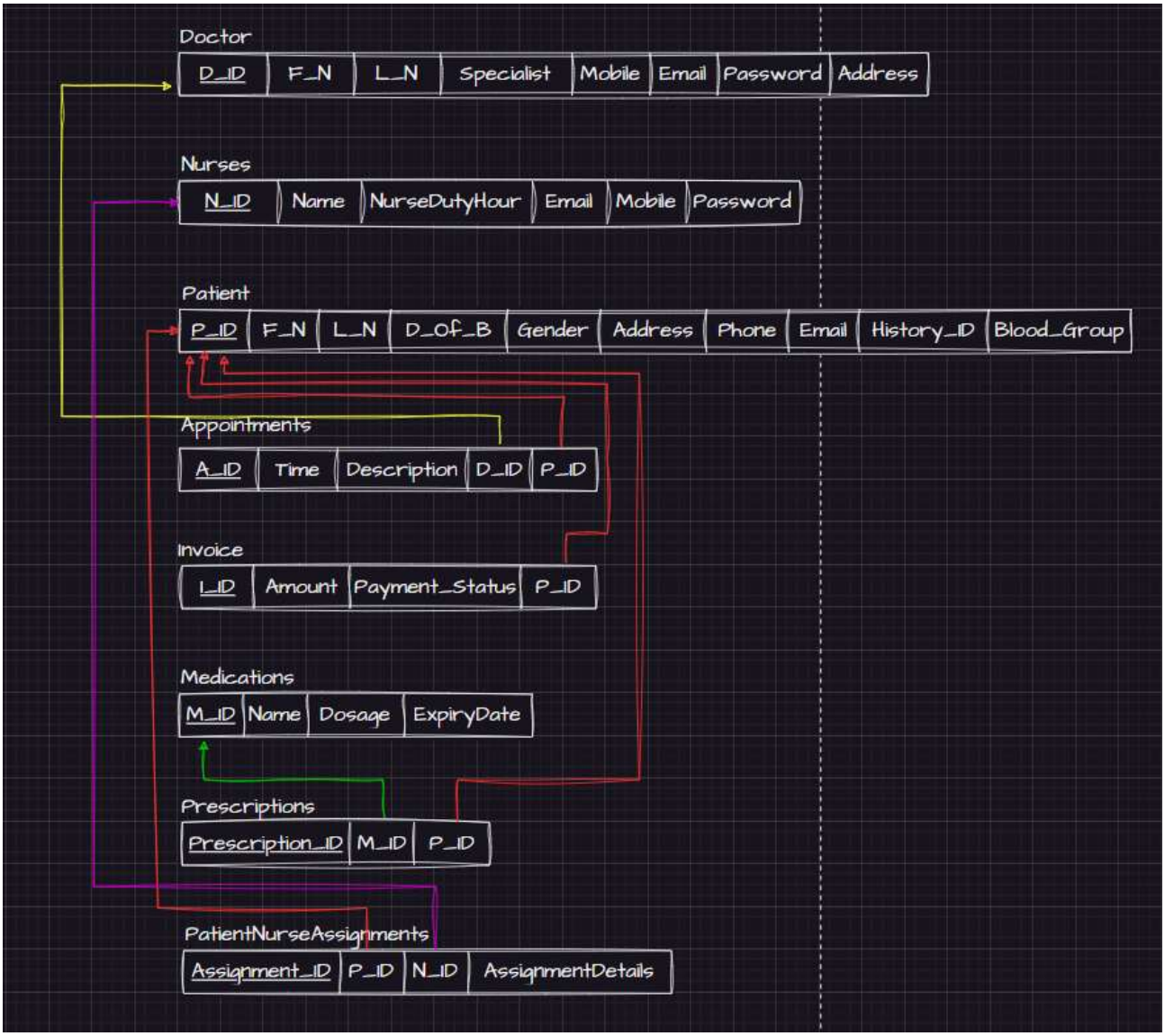
InvoiceID → [PatientID, Amount, PaymentStatus]

PatientID → [Patient Details]

❖ UML for Project



❖ Mapping for Project



BCNF

Upon reviewing each table in our database, we've observed that every non-trivial functional dependency is characterized by having a superkey as its determinant. This key feature ensures that all tables conform to the requirements of Boyce-Codd Normal Form (BCNF).

In the normalization process, we evaluate tables against various normal forms' criteria, specifically looking for opportunities to reduce redundancy and enhance data integrity through structured decomposition. BCNF, as a more rigorous extension of the Third Normal Form (3NF), demands that every determinant in a functional dependency must be a superkey, which our tables satisfy.

Given that all tables meet the BCNF criteria, they maintain the desired level of normalization without necessitating additional decomposition. The current state of the database is such that it upholds the principles of BCNF, indicating no redundancy and a robust structure against common data manipulation anomalies.

Tools used in the project

Netbeans ,PgAdmin , Postgresql , Jaspersoft Studio, draw.io

A description of how the project's GUI works

When the program is initiated, a page appears containing options for 'Login as Admin, Doctor, Nurse, Patient, and Guest'.

Admin: Access is granted upon entering the master key and password. The admin interface includes buttons for adding entities such as doctors, nurses, patients, appointments, medications, and invoices. The admin can view all data related to everything, and has the ability to delete, edit, and search for anything in the database, except for the master key, which cannot be updated. The admin can search using the master key or first name, and edit data through a dedicated table, but cannot modify the master key. Deletion is also performed using the master key.

Guest: Guests can access the system without a master key or password. They can view the names of doctors and nurses, along with their email addresses for contact purposes.

Doctor: Access is granted upon entering the master key and password. Doctors can view patient appointment details and search for specific appointments using the master key. They can enter a patient's master key to view their name.

Patient: Access is granted upon entering the master key and password. Patients can view their medication and invoice data by entering their assigned master key.

Nurse: Access is granted upon entering the master key and password. Nurses can view and search for the data of patients under their care.